

Project 2 of “Neural Network and Deep Learning”

CIFAR-10 Classification and Batch Normalization Analysis

Changhao Jia
23307140027

May 29, 2026

Abstract

This project studies CIFAR-10 image classification and the effect of Batch Normalization (BN) on optimization. We implement three VGG-style models in PyTorch: a plain VGG-A baseline, VGG-A with BN, and a VGG-A variant with Dropout. We compare optimizers and learning rates, run both subset-scale and full-data experiments, and visualize loss-landscape and gradient-stability behavior to analyze how BN affects training. In the final full-data training, plain VGG-A trained with Adam at learning rate 10^{-4} reaches the best test accuracy of 76.83%, while VGG-A with BN reaches 74.60% under the same setting. Although BN does not produce the highest final full-data accuracy in our best tested configuration, it improves optimization stability in the smaller-scale experiments and produces smoother optimization diagnostics.

1 Submission Information

Name: Changhao Jia

Student ID: 23307140027

GitHub repository: <https://github.com/jiab666/Neural-Network-and-Deep-Learning-pj2>

Dataset link: <https://www.cs.toronto.edu/~kriz/cifar.html>

Model weights link: https://drive.google.com/drive/folders/1x8BPofWcaTG0FGQ4PiEKD1DMwgU1n.vf?usp=drive_link

2 Introduction

CIFAR-10 is a standard benchmark for image classification and contains 60,000 32×32 color images from 10 categories [3]. Its low resolution makes it a practical testbed for studying model design and optimization. In this project, the first goal is to train a convolutional neural network that includes convolution, pooling, activation, and fully-connected layers. The second goal is to investigate how Batch Normalization affects training dynamics and optimization behavior.

To satisfy the project requirements, we implement a VGG-style architecture, include Batch Normalization as the main extension component, keep Dropout as an additional comparison, compare multiple optimizers from `torch.optim`, and visualize the loss landscape and gradient behavior. The central question of this report is not only which model achieves the highest final accuracy, but also how BN changes training stability and sensitivity to optimization hyperparameters.

3 Dataset and Experimental Setup

3.1 Dataset

We use CIFAR-10 [3]. The full dataset contains 50,000 training images and 10,000 test images. During the exploratory stage, we also use a subset-scale setup with 5,000 training images and 1,000 test images for faster iteration and easier comparison of optimizers and learning rates.

3.2 Models

We implement three VGG-style models in `codes/VGG_BatchNorm/models/vgg.py`:

- **VGG-A**: plain baseline model.
- **VGG-A + BN**: BN inserted after each convolution and before hidden ReLU activations in the classifier.
- **VGG-A + Dropout**: Dropout inserted in the classifier.

All models contain convolutional layers, max-pooling layers, ReLU activations, and fully-connected layers, which satisfies the basic network requirements of the assignment.

3.3 Training Pipeline

All experiments are implemented in PyTorch. The main training and analysis script is `codes/VGG_BatchNorm/V`. We extend the original skeleton code into a reusable runner that supports:

- model and optimizer comparison,
- automatic saving of training curves and metrics,
- loss-landscape and gradient diagnostics,
- resumable checkpoints for long experiments.

To cover the required optimization strategy, we compare SGD, SGD with momentum, and Adam [2]. The main subset experiments are run for 5 to 12 epochs, while the final full-data training is run for 40 epochs.

4 Results for CIFAR-10 Classification

4.1 Subset-Scale Search Results

We first run small-scale experiments on 5,000 training images and 1,000 test images to verify the code and choose promising settings. Table 1 summarizes representative results.

Table 1: Representative subset-scale results on 5,000/1,000 CIFAR-10 split.

Model	Optimizer	Learning rate	Best accuracy
VGG-A	Adam	10^{-4}	49.40%
VGG-A + BN	Adam	10^{-4}	53.60%
VGG-A + BN	Adam	10^{-3}	47.60%
VGG-A + BN	SGD + momentum	10^{-3}	47.20%
VGG-A + BN	SGD	10^{-3}	37.30%
VGG-A + Dropout	Adam	10^{-4}	11.20%

These results already show two important trends. First, optimizer choice matters substantially: Adam and SGD with momentum are both clearly stronger than plain SGD. Second, BN improves optimization behavior in the exploratory stage. The best subset run of VGG-A with BN reaches 53.6%, outperforming the best plain VGG-A subset run at 49.4%.

The Dropout model performs poorly across the tested learning rates and remains close to random-guess performance. This negative result is still informative: an extra regularization component does not automatically improve optimization or accuracy.

4.2 Final Full-Data Results

After the exploratory stage, we run full-data experiments on the complete CIFAR-10 training set and evaluate on the complete test set. Our main final comparison uses Adam with learning rate 10^{-4} for 40 epochs.

Table 2: Final full-data results on CIFAR-10.

Model	Best test accuracy	Best epoch	Final epoch accuracy
VGG-A	76.83%	31	74.47%
VGG-A + BN	74.60%	30	72.82%

Figure 1 shows the corresponding training curves. In the final full-data configuration, plain VGG-A achieves the strongest final classification result among the tested settings.

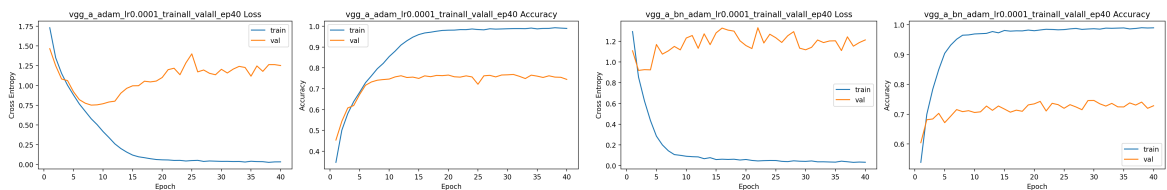


Figure 1: Full-data training curves for VGG-A (left) and VGG-A + BN (right).

5 VGG-A with and without Batch Normalization

To analyze BN more directly, we compare VGG-A and VGG-A + BN under the same backbone design. The BN version is implemented by inserting `BatchNorm2d` after every convolution and `BatchNorm1d` before the hidden classifier activations. This keeps the structure close to the baseline and makes the comparison interpretable.

The comparison leads to a nuanced conclusion:

- In the subset-scale experiments, BN provides a clear optimization advantage and is more robust to learning-rate changes.
- In the final full-data 40-epoch experiment with Adam and learning rate 10^{-4} , the plain VGG-A model slightly outperforms the BN variant in final test accuracy.

Therefore, BN is not universally better in every configuration. Instead, our results suggest that BN is particularly helpful during the early-stage tuning process and in stability-sensitive settings, while the best final classification accuracy still depends on the full training recipe.

6 How Does BN Help Optimization?

Following the project requirement and the analysis in [4, 1], we study BN through loss-landscape and gradient diagnostics. The main idea is that if BN smooths the optimization landscape, then

training should become less sensitive to step size and gradient changes should become more predictable.

6.1 Loss-Landscape Comparison

We train several subset-scale runs with different learning rates ($10^{-3}, 2 \times 10^{-3}, 5 \times 10^{-4}, 10^{-4}$) and track a probe loss across epochs. For each epoch, we take the minimum and maximum loss among these runs and visualize the envelope.

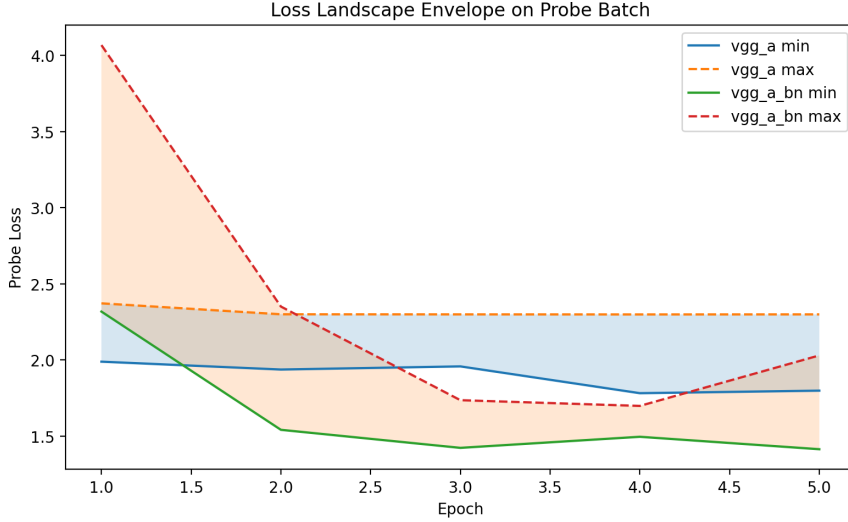


Figure 2: Loss-landscape envelope comparison between VGG-A and VGG-A + BN.

Figure 2 shows that the BN model occupies a more stable region in the subset experiments. This is consistent with the intuition that BN reparameterizes the problem into a smoother landscape and reduces sensitivity to step size.

6.2 Gradient Diagnostics

We also track the probe gradient norm and the gradient difference between consecutive epochs.

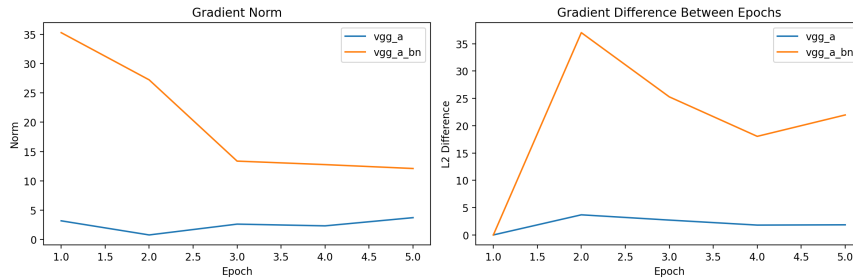


Figure 3: Gradient norm and gradient-difference diagnostics for VGG-A and VGG-A + BN.

Figure 3 indicates that, during the small-scale search stage, the BN model usually changes more smoothly across training than the non-BN model. Thus, even though the final full-data accuracy winner is plain VGG-A in our best tested configuration, the optimization diagnostics still support the claim that BN improves training stability.

7 Discussion

This project leads to three main observations.

Optimizer choice matters. Adam and SGD with momentum are both much stronger than plain SGD in our setup. This effect is especially visible for the BN model.

BN is useful for optimization. During the exploratory stage, BN improves stability and often yields better subset accuracy. It is also more robust to learning-rate changes in the tested range.

Better optimization behavior does not always imply the best final accuracy. Our final full-data run shows that plain VGG-A can still slightly outperform the BN variant in classification accuracy under a fixed recipe. Therefore, the role of BN should be understood as an optimization tool whose benefit depends on the overall configuration.

For future improvement, the most promising directions are stronger data augmentation, a learning-rate schedule, and longer BN-specific tuning. These changes may improve both models and could also change the final ranking between plain VGG-A and VGG-A + BN.

8 Reproducibility Notes

The main source files used in this report are:

- `codes/VGG_BatchNorm/models/vgg.py`
- `codes/VGG_BatchNorm/VGG_Loss_Landscape.py`

The experiment runner supports resumable checkpoints and automatically distinguishes runs by model, optimizer, learning rate, training subset size, validation subset size, and epoch count. This makes it easier to reproduce both exploratory and final full-data experiments.

References

- [1] Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *International Conference on Machine Learning*, pages 448–456, 2015.
- [2] Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.
- [3] Alex Krizhevsky, Geoffrey Hinton, et al. Learning multiple layers of features from tiny images. *Technical Report*, 2009.
- [4] Shibani Santurkar, Dimitris Tsipras, Andrew Ilyas, and Aleksander Madry. How does batch normalization help optimization? In *Advances in Neural Information Processing Systems*, 2018.